Manual Técnico – SkyBalanceAVL

Este manual es una guía completa que explica el funcionamiento del sistema SkyBalanceAVL a nivel técnico. Contiene la documentación detallada de todos los componentes del backend y frontend del proyecto. El lector debe contar con conocimientos básicos de programación.

Requisitos Técnicos para Funcionamiento

Backend (Python): Python 3.7+, Flask 2.0.0+, flask-cors 3.0.10+
Frontend (JavaScript): ES6+ (ES2017), HTML5, Bootstrap 5.3.8
Navegadores: Chrome 55+, Firefox 52+, Safari 10.1+, Edge 15+ (NO compatible: Internet Explorer)
Sistema Operativo: Windows 7 SP1+, macOS 10.12+, Linux (Ubuntu 16.04+, Fedora 23+)

Tabla de Contenidos – Backend

Fase 1: Fundamentos (Modelos y Estructuras)

Node.py - Nodo genérico de árbol binario
Flight.py - Modelo de datos de vuelo
Stack & Queue - Estructuras LIFO/FIFO

Fase 2: Árboles (Algoritmos)

AvlTree.py - Árbol binario balanceado con rotaciones
BstTree.py - Árbol binario simple (referencia)

Fase 3: Servicios (Lógica de Negocio)

VersionService - Control de versiones y snapshots

Fase 4: Validación y Pruebas

StressModeService - Activación de modo estrés y recuperación

Fase 5: Aplicación (Capas de Presentación)

app.py - Inicialización Flask y configuración
TreeController - Endpoints REST API
utils.py - Funciones auxiliares I/O

Tabla de Contenidos – Frontend

Fase 1: Diseño

index.html - Inicio de la aplicación
app.html - Interfaz principal

Fase 2: Implementación

app-main.js - Controlador principal
api-client.js - Cliente HTTP
tree-visualizer.js - Visualizador gráfico

Backend – Documentación

Manual Técnico: Node.py
Archivo: backend/models/Node.py
Tipo: Modelo de Datos

Clase genérica de nodo binario utilizada por tanto AVL Tree como BST Tree. Encapsula un valor Flight y mantiene referencias a nodo padre y nodos hijo.

1. PROPÓSITO GENERAL
Node es la **unidad fundamental de la estructura de árbol binario**. Cada nodo:
- Almacena un valor (objeto Flight)
- Mantiene referencia al nodo padre
- Mantiene referencias a hijo izquierdo y derecho
- Almacena altura (para cálculos AVL)
- Mantiene estado crítico para validaciones

2. ESTRUCTURA Y ATRIBUTOS

Atributo Privado	Tipo	Propósito	Valor Inicial
_value	Flight	Objeto vuelo almacenado en el nodo	Parámetro del constructor
_parent	Node |None	Referencia al nodo padre	None
_leftChild	Node |None	Referencia al hijo izquierdo	None
_rightChild	Node |None	Referencia al hijo derecho	None
_isCritical	bool	Marca si el nodo es crítico(desbalanceado)	False
_height	int	Altura del nodo en el árbol (para balanceo AVL)	0

3. CONSTRUCTOR

```
def __init__(self, value):
    self._value = value
    self._parent = None
    self._leftChild = None
    self._rightChild = None
    self._isCritical = False
    self._height = 0
```

Parámetro: value - Objeto Flight que será almacenado en el nodo
Inicialización: Todos los atributos se inicializan con sus valores por defecto

4. MÉTODOS GETTERS

Método	Retorna	Propósito
getValue()	Flight	Obtiene el vuelo almacenado en el nodo

getHeight()	int	Obtiene la altura actual del nodo
getLeftChild()	Node | None	Obtiene referencia al hijo izquierdo
getRightChild()	Node | None	Obtiene referencia al hijo derecho
getParent()	Node | None	Obtiene referencia al nodo padre
isCritical()	bool	Verifica si el nodo está marcado como crítico

5. MÉTODOS SETTERS

Método	**Parámetro**	**Propósito**
setValue(value)	Flight	Actualiza el vuelo almacenado
setHeight(height)	int	Actualiza la altura del nodo
setLeftChild(node)	Node | None	Asigna el hijo izquierdo
setRightChild(node)	Node | None	Asigna el hijo derecho
setParent(node)	Node | None	Asigna el nodo padre

6. EJEMPLO DE USO

```python
from models.Node import Node
from models.Flight import Flight

# Crear un vuelo
flight = Flight(
    idFlight="AA001",
    departureCity="Bogotá",
    arrivalCity="Medellín",
    departureDate="10:30",
    price=250000,
    numberPassengers=180,
    promotion=False,
    alert=False,
    priority=1
)

# Crear nodo con el vuelo
node = Node(flight)

# Acceder al vuelo
print(node.getValue().getIdFlight())  # Output: "AA001"

# Establecer altura
node.setHeight(0)

# Crear relación padre-hijo
parent_node = Node(parent_flight)
child_node = Node(child_flight)

parent_node.setLeftChild(child_node)
child_node.setParent(parent_node)
```

7. ESTRUCTURA DE ÁRBOL VISUAL

```
        Node(AA001)
        /          \
   Node(XX)      Node(YY)
   /    \        /    \
Node(A) Node(B) Node(C) Node(D)

Relaciones:
- Node(XX).getParent() == Node(AA001) ✓
- Node(AA001).getLeftChild() == Node(XX) ✓
- Node(XX).getRightChild() == Node(B) ✓
- Node(A).getParent() == Node(XX) ✓
```

8. IMPORTANCIA EN AVL vs BST

Para AVL Tree:
La _height es crítica para calcular el factor de balance
Se actualiza después de cada inserción/eliminación
Determina cuándo hacer rotaciones

Para BST Tree:
La _height no se utiliza (no es necesaria)
Solo mantiene la estructura binaria básica
Las relaciones padre-hijo son suficientes

Manual Técnico: Flight.py

Archivo: backend/models/Flight.py
Tipo: Modelo de Datos

Clase que encapsula toda la información de un vuelo de la aerolínea SkyBalance.
Contiene datos de identificación, pasajeros, pricing, horarios y ciudades.

1. PROPÓSITO GENERAL

Flight es el **modelo de datos central** del sistema. Representa un vuelo individual con:
Identificación única (código de vuelo)
Información de ruta (origen/destino)
Información de pasajeros y capacidad
Datos de precios y promociones
Horarios de salida
Estados de alerta y prioridad

2. ATRIBUTOS Y TIPOS DE DATOS

AtributoPrivado	Tipo	Descripción	Valor porDefecto
_idFlight	str	Identificador único del vuelo (ej: AA001,BA002)	None
_numberPassengers	int	Cantidad de pasajeros en el vuelo	0
_promotion	bool	¿El vuelo tiene promoción activa?	False
_price	float	Precio base del ticket en pesos	0.0
_priority	int	Nivel de prioridad (1-3)	0
_departureDate	str	Hora de salida en formato HH:MM	"00:00"
_departureCity	str	Ciudad de origen del vuelo	""
_arrivalCity	str	Ciudad de destino del vuelo	""
_alert	bool	¿Hay alertas para este vuelo?	False

3. CONSTRUCTOR

```
def __init__(self, idFlight=None, numberPassengers=0, promotion=False,
             price=0.0, priority=0, departureDate="00:00",
             departureCity="", arrivalCity="", alert=False):
```

Parámetros: Todos tienen valores por defecto, permitiendo crear vuelos con información parcial
Inicialización: Todos los atributos se asignan directamente al diccionario privado

Ejemplo de uso:

```
flight = Flight(
    idFlight="AA001",
    numberPassengers=180,
    promotion=False,
    price=250000,
    priority=1,
    departureDate="10:30",
    departureCity="Bogotá",
    arrivalCity="Medellín",
    alert=False
)
```

4. MÉTODOS GETTERS

Método	Retorna	Descripción
getIdFlight()	str	Obtiene el ID único del vuelo
getNumberPassengers()	int	Obtiene cantidad de pasajeros
getPromotion()	bool	Verifica si hay promoción
getPrice()	float	Obtiene precio base del ticket
getPriority()	int	Obtiene nivel de prioridad (1-3)
getDepartureDate()	str	Obtiene hora de salida
getDepartureCity()	str	Obtiene ciudad de origen
getArrivalCity()	str	Obtiene ciudad de destino
getAlert()	bool	Verifica si hay alertas

5. MÉTODOS SETTERS CON VALIDACIÓN

5.1 setNumberPassengers(numberPassengers)

```
def setNumberPassengers(self, numberPassengers):
    if numberPassengers < 0:
        raise ValueError("Number of passengers cannot be negative.")
    self._numberPassengers = numberPassengers
```

Validación: Lanza excepción si es negativo

5.2 setPrice(price)

```
def setPrice(self, price):
    if price < 0:
        raise ValueError("Price cannot be negative.")
    self._price = price
```

Validación: Lanza excepción si es negativo

5.3 setPriority(priority)

```
def setPriority(self, priority):
    if priority < 1 or priority > 3:
        raise ValueError("Priority should be between 1 and 3.")
    self._priority = priority
```

Validación: Solo permite 1, 2 o 3

5.4 Otros Setters (sin validación)

setIdFlight(idFlight) - Asigna ID del vuelo
setPromotion(promotion) - Activa/desactiva promoción
setDepartureDate(departureDate) - Actualiza hora de salida
setDepartureCity(departureCity) - Actualiza ciudad origen
setArrivalCity(arrivalCity) - Actualiza ciudad destino
setAlert(alert) - Actualiza estado de alerta

6. COMPARACIÓN ENTRE VUELOS

¿Cómo se comparan dos vuelos en el árbol?
Los vuelos se comparan por su _idFlight usando >, <, ==:

```
flight1 = Flight(idFlight="AA001", ...)
flight2 = Flight(idFlight="BB001", ...)

if flight1.getIdFlight() < flight2.getIdFlight():
    # AA001 < BB001, flight1 va a la izquierda del árbol
```

Orden alfabético/lexicográfico: Los códigos se ordenan como strings

7. CONSIDERACIONES
NOTA SOBRE COMPARACIÓN: Las instancias Flight se comparan en el código del controlador una por una cuando se convierte a JSON, pero en el árbol se compara por el código (ID) del vuelo como strings.

Encapsulación: Todos los atributos son privados (_) para control de acceso
Validación: Los setters validan rangos donde es apropiado
Valores por defecto: Permiten crear vuelos en estados parciales
Sin métodos de cálculo: El pricing y penalizaciones se calculan en servicios separados.

8. INTEGRACIÓN CON EL SISTEMA

En Node: Cada Flight está envuelto en un Node que mantiene referencias del árbol
En AvlTree: Los vuelos se insertan basándose en comparación de getIdFlight()
En Servicios: TreeService usa Flight para crear, actualizar y eliminar registros
En API: El controlador parsea JSON del cliente a objetos Flight

Manual Técnico: Stack.py yQueue.py

Archivos: backend/models/Stack.py, backend/models/Queue.py
Tipo: Estructuras de Datos Auxiliares

Dos estructuras de datos fundamentales de propósito general: Stack (LIFO - LastIn First Out) y Queue (FIFO - First In First Out). Se utilizan para historial de operaciones y procesamiento de vuelos.

1. STACK.PY - PILA (LIFO)

1.1 Propósito General
Stack es una **estructura de datos Last-In-First-Out (LIFO)**. Sus usos en el proyecto:
Almacenar historial de operaciones (insert, delete, update)
Implementar undo/redo funcionalidades
Gestionar estados previos del árbol

1.2 Estructura Interna

```
class Stack:
    def __init__(self):
        self._data = []  # Lista Python como almacenamiento interno
```

Implementación: Usa una lista de Python (array dinámico) interna

1.3 Métodos

Método	Parámetros	Función	Complejidad
push(value)	value: any	Añade elemento al tope de la pila	O(1)

pop()	Ninguno	Extrae y retorna el elemento del tope	O(1)
getTop()	Ninguno	Obtiene el elemento del tope sin extraerlo	O(1)
isEmpty()	Ninguno	Verifica si la pila está vacía	O(1)
getData()	Ninguno	Retorna copia de la lista interna	O(n)

1.4 Manejo de Errores

```
def pop(self):
    if(self.isEmpty()):
        raise IndexError("the stack is empty, you can't pop an element")
    return self._data.pop()
```

Excepción: Se lanza IndexError si se intenta pop/getTop en pila vacía

1.5 Ejemplo de Uso

```
# Crear pila para historial
history = Stack()

# Guardar estados
history.push("insert_AA001")
history.push("insert_BB002")
history.push("insert_CC003")

# Verificar si hay historial
if not history.isEmpty():
    last_operation = history.pop()  # "insert_CC003"

# Ver tope sin extraer
top = history.getTop()  # "insert_BB002"
```


1.6 Visualización LIFO

```
Operaciones en orden:
push("A") → [A]
push("B") → [A, B]
push("C") → [A, B, C]  ← top
pop()     → [A, B]     (retorna C)
pop()     → [A]        (retorna B)
```

2. QUEUE.PY - COLA (FIFO)

2.1 Propósito General

Queue es una **estructura de datos First-In-First-Out (FIFO)**. Sus usos:
Procesar vuelos en orden de llegada
Implementar cola de inserción masiva
BFS (Búsqueda en Amplitud) en árboles

2.2 Estructura Interna

```
from collections import deque

class Queue:
    def __init__(self):
        self._data = deque()  # Doble-ended queue de collections
```

Implementación: Usa deque de Python (más eficiente que lista para operaciones FIFO)

2.3 Métodos

Método	Parámetros	Función	Complejidad
enqueue(value)	value: any	Añade elemento al final de la cola	O(1)
dequeue()	Ninguno	Extrae y retorna el primer elemento	O(1)
getFirst()	Ninguno	Obtiene el primer elemento sin extraerlo	O(1)
isEmpty()	Ninguno	Verifica si la cola está vacía	O(1)
getData()	Ninguno	Retorna copia de la lista interna	O(n)

2.4 Manejo de Errores

```python
def dequeue(self):
    if(self.isEmpty()):
        raise IndexError("the queue is empty, you can't dequeue an element")
    return self._data.popleft()
```

Excepción: Se lanza IndexError al intentar dequeue/getFirst en cola vacía

2.5 Ejemplo de Uso

```python
# Crear cola para procesar vuelos
queue = Queue()

# Encolar vuelos
queue.enqueue("AA001")
queue.enqueue("BB002")
queue.enqueue("CC003")

# Procesar en orden FIFO
while not queue.isEmpty():
    flight = queue.dequeue()
    process_flight(flight)
    # Orden: AA001, BB002, CC003
```

2.6 Visualización FIFO

```
Operaciones en orden:
enqueue("A")  → [A]
enqueue("B")  → [A, B]
enqueue("C")  → [A, B, C]
dequeue()     → [B, C]       (retorna A)
dequeue()     → [C]          (retorna B)
```

Manual Técnico: AvlTree.py

Archivo: backend/models/AvlTree.py
Tipo: Estructura de Datos (Árbol Auto-Balanceado)

Implementación completa de Árbol AVL (Adelson-Velsky and Landis). Un árbol binario de búsqueda auto-balanceado que mantiene diferencia de alturas máximo 1 entre subárboles de cualquier nodo, garantizando operaciones en O(log n).

1. PROPÓSITO Y CARACTERÍSTICAS

AvlTree es la **estructura de datos principal** del sistema SkyBalance. Características:

Auto-balanceado: Se rebalancea automáticamente después de cada inserción/eliminación
Balance factor: Diferencia de alturas entre subárboles ≤ 1 en cualquier nodo
Complejidad O(log n): Búsqueda, inserción y eliminación en tiempo logarítmico
Rotaciones: Implementa 4 tipos (LL, RR, LR, RL) para mantener balance
Metrics: Rastreo de estadísticas de rotaciones

2. CONSTRUCTOR E INICIALIZACIÓN

```
def __init__(self):
    self.root = None
    self.rotations = {
        "LL": 0,  # Rotación izquierda-izquierda
        "RR": 0,  # Rotación derecha-derecha
        "LR": 0,  # Rotación izquierda-derecha
        "RL": 0   # Rotación derecha-izquierda
    }
```

Atributos:
root - Nodo raíz del árbol (None si está vacío)
rotations - Contador de rotaciones por tipo (para métricas)

3. OPERACIONES PRINCIPALES

3.1 INSERT - Inserción de Nodos

Aspecto	Descripción
Firma	insert(self, node, rebalance=True)
Parámetros	node: Node, rebalance: bool
Función	Inserta un nodo y rebalancea automáticamente
Complejidad	O(log n)
Duplicados	No se insertan (rechazo automático)

Proceso:

```
1. Si árbol vacío → nodo es raíz
2. Si no → inserción recursiva
   - Si valor < nodo actual → insertar a la izquierda
   - Si valor > nodo actual → insertar a la derecha
3. Actualizar altura del nodo
4. Rebalancear si es necesario
```

3.2 DELETE - Eliminación de Nodos
Elimina un nodo y rebalancea el árbol.

Tres casos manejados:

Nodo hoja → eliminar directamente
Un hijo → reemplaza su hijo
Dos hijos → encontrar sucesor, copiar valor, eliminar sucesor

3.3 SEARCH – Búsqueda

```
def search(self, value):
    # Búsqueda binaria O(log n)
```

Función: Busca un nodo con un valor específico
Complejidad: O(log n)

3.4 REBALANCE - Rebalanceo
Mantiene la propiedad AVL después de cambios:

```
Factor Balance = altura(izq) - altura(der)

Casos desbalanceados:
- LL (Left-Left): balance > 1, hijo izq tiene balance ≥ 0
- LR (Left-Right): balance > 1, hijo izq tiene balance < 0
- RR (Right-Right): balance < -1, hijo der tiene balance ≤ 0
- RL (Right-Left): balance < -1, hijo der tiene balance > 0
```

4. ROTACIONES
Las 4 operaciones de rotación:

4.1 Single Right Rotation (RR)

```
Antes:          Después:
    A                B
   /               / \
  B       →       C   A
 /
C
```

4.2 Single Left Rotation (LL)

```
Antes:          Después:
  A                B
   \              / \
    B      →     A   C
     \
      C
```

4.3 Left-Right Rotation (LR)

```
Antes:          Paso 1:         Después:
  A               A                B
 /               /               / \
B        →      C        →      C   A
 \              /
  C            B
```

4.4 Right-Left Rotation (RL)

```
Antes:          Paso 1:         Después:
A               A                B
 \               \               / \
  B        →      B       →     A   C
 /                \
C                  C
```

5. MÉTODOS ÚTILES

Método	Función	Complejidad
getRoot()	Obtiene nodo raíz	O(1)
getRotationStats()	Obtiene contador de rotaciones	O(1)
resetRotationStats()	Reinicia contadores a 0	O(1)
cancelationNode(value)	Cancela un nodo y sus descendientes	O(log n)
rebalanceFull(node)	Rebalancea todo un subárbol recursivamente	O(n)

6. EJEMPLO DE USO

```
from models.AvlTree import AvlTree
from models.Node import Node
from models.Flight import Flight

# Crear árbol
avl = AvlTree()

# Crear y insertar nodos
for flight_data in flights:
    flight = Flight(
        idFlight=flight_data["codigo"],
        # ... otros parámetros
    )
    node = Node(flight)
    avl.insert(node)

# Buscar un vuelo
found_node = avl.search(Flight(idFlight="AA001"))

# Eliminar un vuelo
avl.delete(Flight(idFlight="AA001"))

# Obtener estadísticas
stats = avl.getRotationStats()
print(f"Rotaciones LL: {stats['LL']}")
print(f"Rotaciones RR: {stats['RR']}")

# Rebalancear completamente
if root:
    avl.root, _ = avl.rebalanceFull(avl.root)
```

7. CASOS DE USO EN SKYBALANCE

Insertar vuelos: Mantiene orden ordenado y equilibrado

Buscar vuelos: Búsqueda rápida por código
Cancelar vuelo: Mantiene integridad del árbol
Comparar con BST: Demuestra ventajas del balanceo
Visualizar operaciones: Mostrar rotaciones en real-time

Manual Técnico: BstTree.py

Archivo: backend/models/BstTree.py
Tipo: Estructura de Datos (Árbol Binario sin Balanceo)

Implementación clásica de Árbol Binario de Búsqueda (BST) sin auto-balanceo. Se mantiene en paralelo al AVL para fines de comparación y demostración de diferencias de rendimiento.

1. PROPÓSITO Y CARACTERÍSTICAS

BstTree es un árbol binario de búsqueda simple sin mecanismos de auto-balanceo. Propósito en el proyecto:

Comparación educativa: Demostrar diferencias entre BST y AVL
Baseline: Árbol simple para contrastar con el AVL balanceado
Análisis de rendimiento: Mostrar deterioro en peor caso
Visualización: Mostrar mismo conjunto de datos en ambas estructuras

Nota: El BST es más "fácil" de entender, pero mucho menos eficiente que AVL cuando los datos no están balanceados.

2. ESTRUCTURA BÁSICA

```
class BstTree:
    def __init__(self):
        """Inicializa BST vacío"""
        self.root = None

    def getRoot(self):
        """Retorna el nodo raíz"""
        return self.root
```

Simplicidad: No mantiene counters de rotaciones ni alturas balanceadas

3. OPERACIÓN: INSERT

```python
def insert(self, node):
    if self.root is None:
        self.root = node
    else:
        self.__insert(self.root, node)
```

Lógica de Inserción Recursiva:

Condición	Acción
node.value == current.value	Rechazo silencioso (sin duplicados)
node.value > current.value	Insertar a la derecha (recursivo o directo)
node.value < current.value	Insertar a la izquierda (recursivo o directo)

Ejemplo de Inserción:

```
Insertando en orden: 5, 3, 7, 2, 4, 6, 8

Resultado BST:
        5
       / \
      3   7
     / \ / \
    2  4 6  8

Complejidad:
- Mejor caso: O(log n) para datos aleatorios
- Peor caso: O(n) si se insertan en orden = lista enlazada
        1
         \
          2
           \
            3  ← Degenerado a lista
             \
              4
```

4. OPERACIÓN: SEARCH

```python
def search(self, value):
    """Búsqueda binaria clásica"""
    return self.__search(self.root, value)
```

Proceso de búsqueda:

Comparar valor con nodo actual
Si igual → encontrado
Si menor → buscar en subárbol izquierdo
Si mayor → buscar en subárbol derecho
Si nodo es None → no encontrado

5. OPERACIÓN: DELETE

```python
def delete(self, value):
    """Elimina nodo con valor específico"""
    self.root = self.__delete(self.root, value)
```

Tres casos de eliminación:

Caso 1: Nodo Hoja (sin hijos)

```
Antes:        Después:
    5             5
   / \    →      /
  3   7         3
```

Caso 2: Un Hijo

```
Antes:        Después:
    5             5
   / \    →      / \
  3   7         6   7
       \
        6
```

Caso 3: Dos Hijos

```
Antes:          Después:
    5               6
   / \     →      / \
  3   7           3   7
 / \
2   4
```

```
Proceso:
1. Encontrar sucesor en-orden (mínimo del subárbol derecho)
2. Copiar valor del sucesor al nodo a eliminar
3. Eliminar el sucesor recursivamente
```

Método	Propósito	Complejidad
insert(node)	Insertar nodo nuevo	O(log n) promedio, O(n) peor
search(value)	Buscar nodo por valor	O(log n) promedio, O(n) peor
delete(value)	Eliminar nodo	O(log n) promedio, O(n) peor
getRoot()	Obtener raíz	O(1)
getLeftChild()	Obtener hijo izquierdo	O(1)
getRightChild()	Obtener hijo derecho	O(1)

7. EJEMPLO DE USO

```
from models.BstTree import BstTree
from models.Node import Node
from models.Flight import Flight

# Crear BST
bst = BstTree()

# Insertar datos
for flight_data in flights:
    flight = Flight(idFlight=flight_data["codigo"], ...)
    node = Node(flight)
    bst.insert(node)

# Buscar
result = bst.search(Flight(idFlight="AA001"))
if result:
    print(f"Encontrado: {result.getValue().getIdFlight()}")

# Eliminar
bst.delete(Flight(idFlight="AA001"))
```

10. USO EN SKYBALANCE

Árbol de comparación: Se mantiene en paralelo al AVL
Análisis visual: Usuario puede ver ambas estructuras lado a lado

Educación: Demuestra perímetro cuándo y por qué el AVL es mejor
Metrics: Comparar diferencias en operaciones

API endpoint:

```
GET /api/tree → Retorna ambos:
{
    "data": { ... },      // AVL Tree
    "dataBst": { ... }    // BST Tree (para comparación)
}
```

Manual Técnico: ServiciosBackend

Carpeta: backend/services/
Documentación de: versionService, metricsService, penaltyService, auditoryService,profitService, queueService

1. VERSIONSERVICE

Archivo: backend/services/versionService.py
Propósito: Implementar control de versiones del árbol. Permite guardar snapshots del estado actual y recuperarlos posteriormente.

1.1 Constructor e Inicialización

```
class VersionService:
    def __init__(self):
        self.versions = {}  # Diccionario: { "nombre": deepcopy(avlTree) }

    def saveVersion(self, name, tree):
        """Guardar snapshot del árbol con nombre único"""
        if not name:
            raise Exception("Version name is required")
        self.versions[name] = deepcopy(tree)

    def loadVersion(self, name):
        """Recuperar snapshot del árbol"""
        return None if name not in self.versions else deepcopy(self.versions[name])

    def getAllVersions(self):
        """Listar todos los nombres de versiones"""
        return list(self.versions.keys())
```

1.2 Métodos Principales

Método	Parámetros	Retorna	Descripción
saveVersion	name: str, tree: AvlTree	void	Crea deepcopy del árbol con nombre único
loadVersion	name: str	AvlTree | None	Recupera copia del árbol por nombre
getAllVersions	-	list[str]	Lista de nombres de versiones guardadas

1.3 Notas Técnicas

- deepcopy crea copia profunda para evitar referencias compartidas
- Cada versión es independiente - cambios posteriores no afectan versiones guardadas
- Sin límite de versiones almacenadas (considerar memory limit en producción)

2. METRICSSERVICE

Archivo: backend/services/metricsService.py
Propósito: Recopilar estadísticas del árbol AVL (altura, rotaciones, recorridos, etc.).

2.1 Función Principal: getMetrics()

```
def getMetrics(self, treeService):
    avl = treeService.avl
    return {
        "altura": avl.getHeight(avl.root),
        "rotaciones": avl.rotations,
        "cancelacionesMasivas": treeService.totalCancelations,
        "recorridos": {
            "inorder": avl.getInOrderJSON(),
            "preorder": avl.getPreOrderJSON(),
            "postorder": avl.getPosOrderJSON(),
            "bfs": avl.breadthFirstSearchJSON()
        },
        "hojas": avl.countLeaves(avl.root)
    }
```

2.2 Métricas Recopiladas

Campo	Tipo	Significado
altura	int	Altura total del árbol AVL
rotaciones	int	Número de rotaciones realizadas
cancelacionesMasivas	int	Número de cancelaciones descendentes
recorridos	dict	Snapshots en 4 órdenes: inorder, preorder, postorder, BFS
hojas	int	Número de nodos hoja

3. PENALTYSERVICE

Archivo: backend/services/penaltyService.py
Propósito: Marcar nodos como "críticos" (penalizados) si superan profundidad límite en stressMode.

3.1 Métodos

```
def setDepthLimit(self, limit):
    self.depthLimit = limit
    self.applyPenalty()

def applyPenalty(self):
    root = self.treeService.avl.root
    if self.depthLimit is None:
        return
    self._traverseAndApply(root, 0)

def _traverseAndApply(self, node, depth):
    if node is None:
        return
    if depth > self.depthLimit:
        node.setCritical(True)
    else:
        node.setCritical(False)
    # Recorrer recursivamente
```

3.2 Regla de Penalización

```
Si profundidad del nodo > depthLimit → node.setCritical(True)
```

Útil para detectar vuelos a mayor profundidad durante pruebas de estrés.

4. AUDITORYSERVICE

Archivo: backend/services/auditoryService.py
Propósito: Auditar integridad del AVL (validar factores de equilibrio).

4.1 Método: auditAVL()

```
def auditAVL(self):
    if not self.treeService.stressMode:
        raise Exception("Auditory only useful with stress mode")
    report, inconsistentNodes = self._auditNode(self.treeService.avl.root)
    return {"treeReport": report, "inconsistentNodes": inconsistentNodes}
```

4.2 Reporte Devuelto

- treeReport: Árbol de información jerárquica (altura, factor equilibrio, consistencia)
- inconsistentNodes: Lista de nodos con |balance| > 1

Solo funciona en stressMode (lanza excepción si está desactivado)

5. PROFITSERVICE

Archivo: backend/services/profitService.py
Propósito: Encontrar y eliminar el vuelo con menor rentabilidad.

5.1 Algoritmo: deleteLowestProfitFlight()

```
Cálculo de Rentabilidad:
profit = totalRevenue - promoValue
donde promoValue = 100 (si promo existe)

Orden de desempate:
1. Menor profit
2. Mayor profundidad en árbol
3. Mayor código (ID vuelo) lexicográfico
```

```
Resultado:
Cancela rama completa del nodo + retorna código del nodo
```

5.2 Integración

```
Llama a treeService.descendantsCancelation() para cancelar el nodo y sus descendientes.
```

6. QUEUESERVICE

Archivo: backend/services/queueService.py
Propósito: Procesar cola FIFO de vuelos, insertarlos en el árbol y capturar snapshots.

6.1 Operaciones Principales

```
def enqueueFlight(self, flight):
    self.queue.enqueue(flight)

def processQueue(self, treeService):
    steps = []
    while not self.queue.isEmpty():
        flight = self.queue.dequeue()
        node = Node(flight)
        treeService.saveState()
        treeService.avl.insert(node, rebalance=rebalance)
        conflict = abs(balance_factor) > 1
        steps.append({
            "inserted": flight.getIdFlight(),
            "conflict": conflict,
            "tree": treeService.getAvlTreeJson()
        })
    return steps
```

6.2 Ciclo de Procesamiento

```
1. Dequeue: Obtiene vuelo de la cola

2. Create Node: Envuelve en nodo
```

```
3. Save State: Guarda snapshots antes de inserción

4. Insert: Inserta en AVL (con rebalanceo según stressMode)

5. Detect Conflict: Verifica si |BF| > 1

6. Record Step: Captura estado del árbol completo
```

6.3 Retorno

```
Lista de pasos (snapshots) con código del vuelo insertado, flag de conflicto,
y estado actual del árbol.
```

Manual Técnico: StressModeService

Archivo: backend/services/stressModeService.py
Tipo: Servicio de Validación y Estrés

Servicio especializado para activar/desactivar "Modo de Estrés" (Stress Mode). En stressMode, se permite que el árbol se desbalancee sin rebalanceo automático. Al desactivar, realiza rebalanceo global completo.

1. PROPÓSITO GENERAL

StressService implementa un mecanismo dual:
Modo Normal (Por defecto): El AVL mantiene balance automático con cada inserción/eliminación
Modo Estrés (stressMode=True): El AVL permite desbalanceos temporales sin intervención automática

Este modo es útil para:
Evaluar robustez del árbol ante situaciones extremas
Ver qué sucede con inserciones masivas sin rebalanceo
Auditar integridad con AuditoryService
Aplicar penalizaciones a nodos profundos con PenaltyService

2. ESTRUCTURA Y MÉTODOS

2.1 Constructor

```
class StressService:
    def __init__(self, treeService):
        self.treeService = treeService
```

```
2.2 Métodos - activateStress()
```

```
def activateStress(self):
    self.treeService.stressMode = True
```

```
Efecto: Establece flag global. TreeService verifica este flag antes de rebalancear.
```

```
2.3 Métodos - desactivateStress()
```

```
def desactivateStress(self):
    self.treeService.stressMode = False
    return self.rebalanceGlobal()
```

2.4 Método Principal - rebalanceGlobal()

Objetivo: Transformar árbol desbalanceado (tolerado en stressMode) a árbol AVL válido**.**

```
def rebalanceGlobal(self):
    avl = self.treeService.avl
    avl.resetRotationStats()

    # Iterar hasta que no haya cambios
    changed = True
    while changed:
        avl.root, changed = avl.rebalanceFull(avl.root)

    # Garantizar parent root es None
    if avl.root is not None:
        avl.root.setParent(None)

    # Aplicar penalizaciones a nodos profundos
    self.treeService.penaltyService.applyPenalty()
```

```
return {
    "message": "Rebalanceo global aplicado",
    "nodes": avl.rootWeight(),
    "rotations": avl.getRotationStats()
}
```

4. NOTAS TÉCNICAS

resetRotationStats(): Limpia contador de rotaciones antes de rebalanceo
rebalanceFull(): Método del AVL que retorna (nuevo_root, cambio_detectado)
rootWeight(): Cuenta total de nodos en árbol
getRotationStats(): Retorna contador detallado de rotaciones (LL, RR, LR, RL)
changed Flag: Loop continúa hasta que no hay cambios (garantiza convergencia)

5. INTEGRACIÓN CON OTROS SERVICIOS

Servicio	Integración
AuditoryService	Solo funciona en stressMode - audita desbalanceos
PenaltyService	Llamado al final de rebalanceGlobal()
MetricsService	Recopila rotaciones que ocurrieron durante rebalanceo
TreeService	Lee flag stressMode antes de cada operación

Manual Técnico: app.py

Archivo: backend/app.py
Tipo: Aplicación Principal

Servidor Flask que actúa como la aplicación principal del backend. Configura CORS, registra blueprints y proporciona el punto de entrada para ejecutar el servidor.

1. PROPÓSITO GENERAL

app.py es el **archivo de configuración y punto de entrada** de la aplicación backend.
Sus funciones principales son:
Crear e instanciar la aplicación Flask
Configurar CORS (Cross-Origin Resource Sharing) para permitir solicitudes desde el frontend
Registrar blueprints de rutas (controllers)
Ejecutar el servidor development con modo debug

2. IMPORTS Y DEPENDENCIAS

Módulo	Origen	Propósito
Flask	flask	Framework web para crear la aplicación HTTP
CORS	flask_cors	Middleware para habilitar CORS en las rutas
tree_bp	controllers.treeController	Blueprint que contiene todas las rutas deoperaciones de árbol

3. ESTRUCTURA DEL CÓDIGO

3.1 Instanciación de Flask

```
app = Flask(__name__)
```

Descripción: Crea la instancia principal de la aplicación Flask. __name__ asegura que Flask encuentre correctamente la carpeta raíz de la aplicación.

3.2 Configuración CORS

```
CORS(app, resources={r"/api/*": {"origins": "*"}})
```

Desglose:
app - Aplicación Flask donde se aplica CORS
resources={r"/api/*": ...} - Patrón que define qué rutas permiten CORS
r"/api/*" - Regex que coincide con todas las rutas bajo /api/
{"origins": "*"} - Permite solicitudes desde cualquier origen (dominio)
NOTA DE SEGURIDAD: El origen "*" permite solicitudes desde cualquier dominio. En producción, debe especificarse un dominio permitido específico:

```
CORS(app, resources={r"/api/*": {"origins": "https://miapp.com"}})
```

3.3 Registro de Blueprint

```
app.register_blueprint(tree_bp, url_prefix="/api/tree")
```

Explicación:

tree_bp - Objeto Blueprint que contiene todas las rutas del árbol
url_prefix="/api/tree" - Prefijo que se antepone a todas las rutas del blueprint
Resultado: Una ruta definida como @tree_bp.route("") se accede como http://localhost:5000/api/tree

3.4 Función Main

```
if __name__ == "__main__":
    app.run(debug=True)
```

Condicional: if __name__ == "__main__" asegura que el servidor solo se ejecute cuando el archivo es ejecutado directamente (no cuando es importado).

app.run(debug=True):

debug=True - Activa el modo debug de Flask
El servidor reinicia automáticamente al detectar cambios en el código
Muestra errores detallados en el navegador
Activa el debugger interactivo
URL por defecto: http://localhost:5000

4. FLUJO DE EJECUCIÓN

```
Inicio
  ↓
1. Crear instancia Flask (app = Flask(__name__))
  ↓
2. Aplicar configuración CORS
  ↓
3. Registrar blueprint tree_bp con prefijo /api/tree
  ↓
4. Verificar if __name__ == "__main__"
  ↓
5. Ejecutar app.run(debug=True)
  ↓
6. Servidor escucha en http://localhost:5000
  ↓
7. El frontend puede hacer solicitudes a http://localhost:5000/api/tree/*
```

Manual Técnico: treeController.py

Archivo: backend/controllers/treeController.py
Tipo: Controlador / Blueprint

Controlador Flask que define todas las rutas REST API para operaciones con árboles AVL. Actúa como intermediario entre las solicitudes HTTP del frontend y la lógica de negocio de TreeService.

1. PROPÓSITO GENERAL

treeController.py es el **controlador principal de rutas API**. Sus responsabilidades:

Definir endpoints REST para operaciones del árbol AVL
Procesar solicitudes HTTP (GET, POST, PUT, DELETE)
Validar y parsear datos JSON del cliente
Invocar servicios de lógica de negocio
Retornar respuestas JSON formateadas
Manejar excepciones y errores

2. IMPORTS Y DEPENDENCIAS

Módulo	Función
Blueprint, jsonify, request	Componentes de Flask para rutas y manejo de JSON
AvlTree, BstTree	Modelos de estructuras de datos
TreeService	Servicio con lógica de negocio para operaciones de árbol
Flight	Modelo de vuelo

3. INICIALIZACIÓN

```
tree_bp = Blueprint("tree", __name__)
service = TreeService()
stressService = StressService(service)
```

Explicación:
tree_bp - Blueprint para todas las rutas de árbol (sin prefijo; agregado en app.py)
service - Instancia de TreeService compartida por todos los endpoints
stressService - Servicio para operaciones en modo estrés

4. ENDPOINTS PRINCIPALES

4.1 GET /tree - Obtener Árbol Actual

```
@tree_bp.route("", methods=["GET"])
def get_tree():
```

Propiedad	Valor
Ruta	GET /api/tree
Propósito	Obtener la estructura actual del árbol AVL en formato JSON
Parámetros	Ninguno
Respuesta Exitosa	200 - JSON con estructura completa del árbol
Respuesta Error	400 - Mensaje de error en JSON

4.2 GET /tree/cities - Obtener Ciudades

```
@tree_bp.route("/cities", methods=["GET"])
def get_cities():
```

Función: Retorna la lista de ciudades con aeropuertos del archivo JSON local.
Ruta: GET /api/tree/cities

4.3 POST /tree/create - Crear Árbol

```
@tree_bp.route("/create", methods=["POST"])
def create_tree():
```

Propiedad	Detalle
Ruta	POST /api/tree/create
Propósito	Crear un nuevo árbol desde datos JSON (2 modos: INSERCION oTOPOLOGIA)
Bodyesperado	{"tipo": "INSERCION", "vuelos": [...]} o {"tipo": "TOPOLOGIA", ...}

Respuesta	201 - Contiene data (AVL) y dataBst (BST para comparación)

4.4 DELETE /tree/cancel/ - Cancelar Subárbol

```
@tree_bp.route("/cancel/", methods=["DELETE"])
def cancel_subtree(flightCode):
```

Propósito: Elimina un nodo y TODOS sus descendientes del árbol.
Parámetro: flightCode - Código del vuelo a cancelar

4.5 POST /tree/insert - Insertar Vuelo

```
@tree_bp.route("/insert", methods=["POST"])
def insert_flight():
```

Propósito: Insertar un nuevo vuelo en el árbol AVL.
Body: Objeto Flight completo en JSON

4.6 GET /tree/search/ - Buscar Vuelo

```
@tree_bp.route("/search/", methods=["GET"])
def search_flight(flightCode):
```

Propósito: Buscar un vuelo por su código.
Retorna: Objeto del nodo encontrado o error 404

4.7 DELETE /tree/delete/ - Eliminar Vuelo

```
@tree_bp.route("/delete/", methods=["DELETE"])
def delete_flight(flightCode):
```

Propósito: Eliminar un nodo individual (sin descendientes).

4.8 PUT /tree/update/ - Actualizar Vuelo

```
@tree_bp.route("/update/", methods=["PUT"])
def update_flight(flightCode):
```

Lógica: Elimina el vuelo antiguo e inserta uno nuevo con los datos actualizados.

5. PATRÓN DE RESPUESTA ESTÁNDAR

```
{
    "status": "success" | "error",
    "data": {...},
    "message": "Descripción"
}
```

Códigos HTTP utilizados:

200 - GET/DELETE exitoso

201 - POST/PUT exitoso (recurso creado)

400 - Error en la solicitud (datos inválidos)

404 - Recurso no encontrado

6. MANEJO DE ERRORES

Patrón utilizado en todos los endpoints:

```
try:
    # Operación
    return jsonify({...}), 200
except Exception as e:
    return jsonify({"status": "error", "message": str(e)}), 400
```

Todos los errores son capturados y retornados como JSON con código 400.

7. FLUJO DE SOLICITUD - EJEMPLO: INSERT

```
Cliente (Frontend)
    ↓ POST /api/tree/insert
        ↓
    treeController.insert_flight()
        ↓ request.get_json()
        ↓
    Crear objeto Flight
        ↓
    service.insertFlight(flight)
        ↓ [TreeService realiza inserción]
        ↓
    service.getAvlTreeJson()
        ↓ [Convierte árbol a JSON]
        ↓
    return jsonify({data: tree})
        ↓
    Respuesta HTTP 201
        ↓
    Cliente recibe JSON
```

8. CONSIDERACIONES DE DISEÑO

Stateful: El árbol persiste en memoria durante la sesión
Sin autenticación: Cualquiera puede modificar el árbol global
Sincrónico: Las respuestas son inmediatas (no hay colas de procesamiento)
JSON como formato: Facilita comunicación frontend-backend
Validación en servicios: La validación ocurre en TreeService, noen el controller

Manual Técnico: utils.py

Archivo: backend/utils/utils.py
Tipo: Funciones Utilitarias

Módulo con funciones auxiliares de propósito general para lectura/escritura de archivos JSON y otras operaciones repetitivas. Centraliza operaciones comunes para evitar duplicación de código.

1. PROPÓSITO GENERAL
utils.py provee **funciones auxiliares reutilizables**:
Lectura de archivos JSON
Escritura de archivos JSON
Adición de contenido a JSON existente
Funciones potenciales para patrones iterativos y operaciones comunes

2. ESTRUCTURA DEL MÓDULO

```
from pathlib import Path
import json

# Ruta interna para almacenamiento por defecto
jsonRouteIntern = Path(__file__).resolve().parent.parent / "data" / "tree.json"
```

Nota: jsonRouteIntern se define pero no se usa actualmente en el código

3. FUNCIONES PRINCIPALES

3.1 readJson(route)

```
def readJson(route):
    with open(route, "r", encoding="utf-8") as archive:
        return json.load(archive)
```

Parámetro	Tipo	Descripción
route	str | Path	Ruta al archivo JSON a leer

Retorna: Objeto Python (dict, list, etc.) parsed desde JSON

Encodificación: UTF-8 (soporta caracteres especiales)

Excepciones: FileNotFoundError, JSONDecodeError

Ejemplo de uso:

```
from utils.utils import readJson
from pathlib import Path

# Leer ciudades
cities_path = Path(__file__).resolve().parent.parent / "data" / "ciudadesConAeropuerto.json"
cities = readJson(cities_path)
print(cities)  # Retorna lista de ciudades
```

3.2 writeJson(jsonData, route)

```
def writeJson(jsonData, route):
    print(f"jsonData: {jsonData}")
    with open(route, "w") as archive:
        archive.write(jsonData)
        print(f"Content written to {route}")
```

Parámetro	Tipo	Descripción
jsonData	str	String JSON ya serializado a escribir
route	str | Path	Ruta de destino del archivo

Información: Sobrescribe el archivo completo (trunca contenido anterior)

Debugging: Imprime en consola para verificación

3.3 addContentToJson(newContent, route)

```
def addContentToJson(newContent, route):
    if newContent == "" or newContent is None:
        return
    print(f"newContent: {newContent}")
    with open(route, "a") as archive:
        newContent = f",{newContent}"
        archive.write(str(newContent))
        print(f"Content added to {route}")
```

Parámetro	Tipo	Descripción
newContent	str	Contenido JSON a añadir (sin la coma inicial)
route	str | Path	Ruta del archivo a modificar

Nota: Este método asume archivo JSON ya existente en formato array

Comportamiento:

Si newContent es vacío o None → no hace nada

Prepend una coma antes del contenido

Append al final del archivo (modo "a")

Útil para arrays JSON donde se quiere agregar elementos

4. USOS EN EL SISTEMA

4.1 En treeController.py

```
from utils.utils import readJson
from pathlib import Path

@tree_bp.route("/cities", methods=["GET"])
def get_cities():
    cities = (
        Path(__file__).resolve().parent.parent
        / "data"
        / "ciudadesConAeropuerto.json"
    )
    data = readJson(cities)
    return jsonify({"status": "success", "data": data}), 200
```

4.2 En servicios auxiliares

Los servicios pueden usar estas funciones para persistencia de datos

5. CONSIDERACIONES DE SEGURIDAD

Path Traversal: Asegurar que route no contenga "../" malicioso

Encoding: UTF-8 soporta caracteres especiales pero considerar XSS si se expone al frontend
Permisos archivos: Verificar que el proceso tiene permiso write/read
Tamaño: No hay límite de tamaño de archivo - archivos enormes pueden causar memory overflow

Frontend – Documentación

Manual Técnico: index.html

Archivo: frontend/index.html

Página de inicio y punto de entrada de la aplicación SkyBalance- Landing page responsivo con navegación principal

1. PROPÓSITO GENERAL

index.html es la **página de inicio y punto de entrada** de la aplicación SkyBalance AVL. Funciona como el **landing page** que presenta la aplicación a los usuarios y proporciona navegación hacia los tres destinos principales:
Ingresar al Sistema → Acceso a app.html (interfaz principal)
Documentación → Enlace a documentación
Acerca de → Página de información del proyecto

2. DEPENDENCIAS REQUERIDAS

```
Debe cargarse DESPUÉS de estos estilos:

        Archivo                          Función
 css/styles.css         Estilos globales responsive (header, hero, botones)
 css/main.css (opcional) Estilos específicos del layout (si se proporcionan)
 Bootstrap 5.3.8        CDN para componentes base, grid system, utilities

Orden de carga en el HTML:

<link rel="stylesheet" href="css/styles.css">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">
```

3. ESTRUCTURA DEL DOM

```
Elementos principales
index.html
├── <header class="header">
│   ├── <img class="header-logo"> — Logo de Universidad de Caldas
│   └── <nav class="header-nav">
│       ├── <button id="btn-documentation"> — Documentación
│       └── <button id="btn-about"> — Acerca de
│
└── <main class="main-content">
    └── <div class="hero">
        ├── <img class="logo-main"> — Logo de SkyBalance
        ├── <h1 class="title"> — "SkyBalance"
        ├── <div class="info">
        │   ├── Desarrolladores: Simón, Nico, Javi
        │   └── Institución: Universidad de Caldas
        └── <button id="btn-enter" class="btn-enter"> — Ingresar al Sistema
```

4. ELEMENTOS INTERACTIVOS

Botones principalesID	Clase	Texto	Destino/Función
#btn-enter	btn-enter	"Ingresar al Sistema"	Redirige a app.html
#btn-documentation	btn-nav	"Documentación"	Redirige a documentation.html
#btn-about	btn-nav	"Acerca de"	Redirige a about.html

5. FUNCIONALIDADES (app.js)

Función: Ingresar al Sistema

```
document.getElementById('btn-enter').addEventListener('click', () => {
    window.location.href = 'app.html';
});
```

Propósito: Navegar hacia la interfaz principal.

Trigger: Click en botón "Ingresar al Sistema"

Flujo: Usuario hace click → Navega a app.html

Función: Acerca de

```
document.getElementById('btn-about').addEventListener('click', () => {
    window.location.href = 'about.html';
});
```

Propósito: Navegar hacia la página de información.

Trigger: Click en botón "Acerca de" en header

Función: Documentación (En desarrollo)

```
document.getElementById('btn-documentation').addEventListener('click', () => {
    window.location.href = 'documentation.html';
});
```

Propósito: Placeholder para acceso a documentación.

Trigger: Click en botón "Documentación" en header

6. FLUJO DE NAVEGACIÓN

```
index.html (Landing Page)
        ↓ [Click "Ingresar al Sistema"]
    app.html (Aplicación Principal)

index.html
        ↓ [Click "Acerca de"]
    about.html (Página de Información)

index.html
        ↓ [Click "Documentación"]
    documentation.html (Página de Documentación)
```

Manual Técnico: app.html

Archivo: frontend/app.html

Interfaz principal de gestión del sistema SkyBalance AVL con layout de 3 columnas (controles | canvas | métricas)

1. PROPÓSITO GENERAL

app.html es la **interfaz principal de la aplicación**, donde los usuarios interactúan con el árbol AVL. Proporciona:
Panel izquierdo: Controles y formularios (320px)
Panel central: Canvas para visualización (flexible)

Panel derecho: Métricas y búsqueda (320px)
Layout: Grid de 3 columnas responsivo

2. PANEL IZQUIERDO

Secciones principales

1 Cargar Árbol

- Button: "Seleccionar archivo JSON"
- Input file (hidden): aceptar .json
- Div: Mostrar nombre del archivo cargado

2 Editar Vuelo

- Input: Código del vuelo a editar
- Button: "Cargar Vuelo"
- Button: "Cancelar Edición" (hidden por defecto)
- Div: Indicador visual de modo edición

3 Insertar Vuelo (formulario principal)

```
<form id="form-insert">
  ├─ Input: código (required)
  ├─ Select: origen (required)
  ├─ Select: destino (required)
  ├─ Input: hora (time, value="10:00")
  ├─ Input: precio (number, min="0")
  ├─ Div: pasajeros-container
  ├─ Input: prioridad (1-3)
  ├─ Checkboxes: promocion, alerta
  └─ Button: submit "Insertar"
</form>
```

4 Programar Vuelo

- Button: "Agregar a Cola"
- Div: Contador "X vuelos en cola"
- Div: Lista de vuelos en cola (scrollable, max 120px)
- Button: "Procesar Cola" (disabled si cola vacía)
- Button: "Limpiar Cola" (disabled si cola vacía)

5 Panel de Procesamiento (hidden)

- Aparece cuando se inicia procesamiento de cola
- Muestra "Paso X/Y: Inserting CODE"
- Muestra métricas del paso actual
- Button: "Detener Procesamiento"

6 Eliminar/Cancelar

- Input: Código del vuelo a eliminar
- Button: "Eliminar Nodo" (solo el nodo)
- Button: "Cancelar Vuelo + Descendientes"
- Button: "Eliminar Nodo Menor Rentabilidad"
- Button: "Eliminar Todo El Árbol"

7 Acciones

- Button: "Deshacer (Ctrl+Z)"
- Button: "Exportar JSON"

8 Modo Estrés

- Toggle: "Modo Estrés"
- Button: "Rebalancear" (disabled si no está en estrés)
- Button: "Verificar Propiedad AVL" (disabled si no está en estrés)

9 Profundidad Crítica

- Input number: límite (default 2, min 1)
- Button: "Actualizar"

10 Versiones

- Input: Nombre de versión
- Button: "Guardar Versión"
- Div: Lista dinámica de versiones (botones "Cargar")

3. PANEL CENTRAL (Flexible)

Estructura

```
<section class="panel panel-center">
  ├─ <div class="canvas-header">
  │    ├─ <h2>Árbol AVL</h2>
  │    └─ <div class="canvas-controls">
  │         ├─ Button: id="btn-zoom-in" (🔍+)
  │         ├─ Button: id="btn-zoom-out" (🔍-)
  │         └─ Button: id="btn-reset-view" (🔄)
  │
  └─ <div class="canvas-container">
       ├─ <canvas id="tree-canvas"></canvas>
       └─ <div class="empty-state"> (hidden)
</section>
```

Características

- <canvas> ocupa 100% del contenedor
- empty-state se muestra cuando currentTree === null
- Controles de zoom en esquina superior derecha
- Renderizado automático al cambiar árbol

4. PANEL DERECHO

Búsqueda de Vuelo

- Input: Código del vuelo
- Button: "Buscar"
- Div: Card con detalles de vuelo (hidden si no encuentra)
- Div: "Vuelo no encontrado" (hidden si encuentra)

Métricas del Árbol

Métrica	ID	Valor por defecto
Altura	#metric-altura	"-" (no hay árbol)
Nodos	#metric-nodos	0
Hojas	#metric-hojas	0

Rotaciones

Tipo	ID	Descripción
LL	#metric-rotacion-ll	Rotación izquierda-izquierda
RR	#metric-rotacion-rr	Rotación derecha-derecha
LR	#metric-rotacion-lr	Rotación izquierda-derecha
RL	#metric-rotacion-rl	Rotación derecha-izquierda

Recorridos

- Button: "InOrder" (#btn-traversal-inorder)
- Button: "PreOrder" (#btn-traversal-preorder)
- Button: "PostOrder" (#btn-traversal-postorder)
- Button: "BFS" (#btn-traversal-bfs)
- Div: Resultado mostrando orden de códigos

Leyenda

Color	Significado
▒	Normal
▒	Promoción
▒	Alerta
▒	Crítico

Versiones

- Input: Nombre descriptivo de versión
- Button: "Guardar Versión"
- Div: Lista dinámica con botones "Cargar"

5. HEADER

```
<header class="app-header">
  ├─ <div class="header-left">
  │   ├─ <img class="app-logo"> (35px)
  │   └─ <h1 class="app-title">SkyBalance AVL</h1>
  │
  ├─ <div class="header-center">
  │   └─ <span id="mode-indicator">Modo Normal</span>
  │
  └─ <div class="header-right">
      └─ <button id="btn-home">Volver</button>
</header>
```

Mode Indicator:

- Gris claro: "Modo Normal"
- Rojo: "Modo Estrés" (cuando stressModeActive=true)

6. TABLA DE BOTONES PRINCIPALES

ID	Evento	Descripción	Condición
#btn-load-file	click	Abre selector de archivo JSON	—
#btn-load-flight	click	Cargar vuelo en modo edición	—
#form-insert	submit	Insertar o actualizar vuelo	Validación HTML
#btn-enqueue	click	Agregar vuelo a cola	—
#btn-process-queue	click	Procesar todos los vuelos encola	queue.length > 0
#btn-delete	click	Eliminar nodo específico	—
#btn-undo	click	Deshacer última acción	—
#btn-zoom-in	click	Ampliar canvas 20%	—
#btn-search-flight	click	Buscar y mostrar detalles devuelo	—
#btn-save-version	click	Guardar versión actual delárbol	input-version-name ≠ ""
#btn-rebalance	click	Rebalancear manualmente	stressModeActive = true
#btn-verify-avl	click	Verificar propiedades AVL	stressModeActive = true

7. CARGA DE SCRIPTS

```
<script src="js/api-client.js"></script>
<script src="js/tree-visualizer.js"></script>
<script src="js/app-main.js"></script>
Orden IMPORTANTE:

1. api-client.js → Declara funciones fetch

2. tree-visualizer.js → Declara clase TreeVisualizer

3. app-main.js → Usa ambas anteriores
```

Manual Técnico: app-main.js

Archivo: frontend/js/app-main.js

Módulo controlador principal del frontend que orquesta todas las interacciones de usuario entre la interfaz HTML y el backend Flask.

1. PROPÓSITO GENERAL

app-main.js es el módulo controlador principal del frontend de la aplicación SkyBalance AVL. Orquesta todas las interacciones de usuario entre la interfaz HTML y el backend Flask.

Maneja:
Operaciones del árbol AVL (inserción, eliminación, búsqueda)
Visualización en canvas mediante el componente TreeVisualizer
Cola de procesamiento concurrente
Métricas y auditoría AVL
Gestión de versiones y exportación

2. DEPENDENCIAS REQUERIDAS

```
Debe cargarse DESPUÉS de estos scripts en el HTML:

      Archivo                              Función
 api-client.js      Comunicación HTTP con el backend Flask
 tree-visualizer.js Clase TreeVisualizer para renderizar árboles en canvas
```

3. ESTADO GLOBAL (VARIABLES GLOBALES)

```
// Variables de árbol
let visualizer = null;           // Instancia de TreeVisualizer
let currentTree = null;          // Último árbol AVL recibido del backend
let stressModeActive = false;    // Flag de modo estrés
let lastLoadedJson = null;       // BST sin balancear (para comparación)

// Variables de cola de inserción (Requisito 3)
let flightQueue = [];            // Array de vuelos pendientes
let processingQueue = false;     // Flag de procesamiento activo
let stopProcess = false;         // Flag para cancelar procesamiento

// Variables de edición
let flightInEdit = null;         // Vuelo en edición activa
```

4. INICIALIZACIÓN (DOMContentLoaded)

Se ejecuta al cargar el DOM. Inicializa:

1. TreeVisualizer en canvas #tree-canvas
2. Carga el árbol del backend
3. Lista de versiones disponibles
4. Ciudades con aeropuerto
5. Registra listeners para 30+ botones y campos del formulario

Flujo de inicialización:

DOMContentLoaded → new TreeVisualizer() → loadTree() → updateVersionList() → loadCities() → addEventListener() × 30+ elementos

5. FUNCIONES PRINCIPALES POR SECCIÓN

5.1 CARGAR/ACTUALIZAR ÁRBOL

Función	Parámetros	Retorna	Descripción
loadTree()	—	Promise<void>	Obtiene el árbol actual del backend. Ejecuta GET /tree → Actualiza currentTree → Ejecuta updateUI()
updateUI()	—	Promise<void>	Redibuja la interfaz completa. Usa requestAnimationFrame para esperar al layout del canvas
showEmptyState()	—	void	Oculta canvas, muestra estado vacío
hideEmptyState()	—	void	Muestra canvas, oculta estado vacío

Nota crítica sobre canvas: El canvas puede tener dimensiones 0 si estaba oculto (display:none). Por eso requestAnimationFrame espera a que el browser calcule el layout real antes de redimensionar.

5.2 CARGAR ARCHIVO JSON

Función	Parámetros	Retorna	Descripción
handleFileSelected(e)	Event	Promise<void>	Cuando usuario selecciona archivo JSON. Valida tipo (INSERCION/TOPOLOGIA) → Envía a backend POST /create → Actualiza árbol
showComparisonModal(treeAVL, treeBST)	Tree, Tree	void	Crea modal con dos canvas lado a lado para comparación AVL vs BST

5.2 CARGAR ARCHIVO JSON

Función	Parámetros	Retorna	Descripción
handleFileSelected(e)	Event	Promise<void>	Cuando usuario selecciona archivo JSON. Valida tipo (INSERCION/TOPOLOGIA) → Envía a backend POST /create → Actualiza árbol
showComparisonModal(treeAVL, treeBST)	Tree, Tree	void	Crea modal con dos canvas lado a lado para comparación AVL vs BST

5.3 INSERCIÓN DE VUELOS

Función	Parámetros	Retorna	Descripción
handleInsertion(e)	FormEvent	Promise<void>	Maneja submit del formulario. Si flightInEdit ≠ null: ejecuta PUT. Si no: ejecuta POST /flights
clearForm()	—	void	Limpia formulario. Si hay vuelo en edición, cancela edición

5.4 EDITAR VUELOS

Función	Descripción
handleLoadFlight()	Busca vuelo por código → Carga datos en formulario → Activa "modo edición"
showPassengersUI(currentPassengers)	Renderiza interfaz mejorada de pasajeros (Agregar/Eliminar con preview)
cancelEdit(isExplicit)	Cancela modo edición, restaura formulario a estado inicial

5.5 ELIMINACIÓN/CANCELACIÓN

Función	Descripción
handleDeletion()	Elimina solo el nodo indicado. Backend reorganiza hijos usando predecesor inorden
handleCancellation()	Elimina nodo + TODA su subrama (más agresivo)
handleDeleteLowestProfit()	Calcula rentabilidad y elimina nodo de menor rentabilidad + subrama
handleDeleteAll()	Elimina árbol completo

5.6 DESHACER/EXPORTAR

Función	Descripción
handleUndo()	Deshace última acción. Ejecuta POST /undo → Restaura árbol anterior
handleExport()	Exporta árbol actual como JSON. Descarga con nombre: skybalance-YYYY-MM-DD.json

5.7 MODO ESTRÉS

Función	Descripción
handleStressMode(e)	Activa/desactiva modo estrés. Cuando se activa: deshabilita balanceo automático. Cuando se desactiva: rebalancea automáticamente
handleRebalance()	Rebalancea árbol manualmente (solo en modo estrés). Devuelve nodos rebalanceados y rotaciones
handleAVLAudit()	Audita propiedades AVL (solo en modo estrés). Identifica nodos inconsistentes donde $\lvert BF \rvert \notin \{-1, 0, 1\}$
showAuditModal(response)	Renderiza modal con resultado de auditoría. Color verde si OK, rojo si hay problemas

Balance Factor (BF):
- AVL válido: $BF \in \{-1, 0, 1\}$
- Inconsistente: $\lvert BF \rvert > 1$

5.8 RECORRIDOS

Función	Descripción
handleTraversal(type)	Obtiene recorrido precomputado del backend (inorder, preorder, postorder, bfs). Extrae códigos de vuelos → Renderiza orden visual

5.9 BÚSQUEDA DE VUELOS

Función	Descripción
handleFlightSearch()	Busca vuelo y muestra detalles: código, origen, destino, hora, pasajeros, precios, prioridad, altura, BF, estado crítico, badges

5.10 MÉTRICAS

Función	Descripción
updateMetrics()	Actualiza panel de métricas. Extrae: altura, nodos, hojas, rotaciones (LL+RR+LR+RL)
countNodes(node)	Recorre árbol recursivamente: return 1 + countNodes(izq) + countNodes(der)
clearMetrics()	Limpia panel de métricas (muestra "-" o "0")

5.11 PROFUNDIDAD CRÍTICA

Función	Descripción
handleUpdateDepth()	Actualiza límite de profundidad crítica. Nodos en profundidad > límite se marcan como críticos

5.12 VERSIONES

Función	Descripción
handleSaveVersion()	Guarda versión del árbol actual con nombre personalizado
updateVersionList()	Obtiene lista de versiones guardadas → Renderiza botones "Cargar" para cada una
handleLoadVersion(name)	Restaura versión anterior → Reemplaza currentTree → Actualiza UI

5.13 COLA DE INSERCIÓN (REQUISITO 3)

Sistema de simulación de concurrencia:

Función	Descripción
getFlightDataFromForm()	Extrae datos del formulario como objeto vuelo. Normaliza: trim(), toUpperCase()
handleEnqueueFlight()	Agrega vuelo a cola local. Valida obligatorios → POST /queue/enqueue → Actualiza UI
updateQueueCounter()	Actualiza contador visual. Habilita/deshabilita botones según haya elementos
updateQueueList()	Renderiza lista visual de vuelos en cola con botones para remover
removeFromQueue(index)	Remueve vuelo específico del índice
handleClearQueue()	Limpia toda la cola
handleProcessQueue()	Procesa cola completa. POST /queue/process → Recibe pasos de inserción → Visualiza cada paso en canvas
handleStopProcessing()	Cancela procesamiento activo
visualizeConcurrencySteps(steps)	Anima inserción paso a paso. Espera 1.5s entre steps. Si desbalance: marca nodos en rojo

Flujo de procesamiento:

Usuarios agregan vuelos → handleProcessQueue() → POST /queue/process → Recibe steps[] → Para cada step: dibuja + espera 1.5s → Resumen final → Resetea cola

5.14 CIUDADES CON AEROPUERTO

Función	Descripción
loadCities()	Carga lista de ciudades colombianas con aeropuerto. GET /cities
populateCitySelects(cities)	Puebla selectores origen/destino con opciones

5.15 UTILIDADES

Función	Descripción
setLoading(btn, asset, text)	Maneja estado de carga de botón. Previene double-clicking durante fetch. Si asset=true: deshabilita. Si asset=false: rehabilita
showToast(message, type, duration)	Notificación tipo toast en esquina superior derecha. Tipos: success (✓), error (✗), warning (⚠), info (ℹ). Duración por defecto: 3.5 segundos

6. VARIABLES DE ESTADO IMPORTANTE

Variable	Tipo	Gestión
visualizer	TreeVisualizer	Inicializa en DOMContentLoaded, nunca se reasigna
currentTree	Object (TreeNode)	Se actualiza en cada operación (insert, delete, undo, load)
stressModeActive	boolean	Toggle en handleStressMode, controla autobalancing
lastLoadedJson	Object	Guarda BST original para comparación AVL vs BST
flightQueue	Array<Flight>	Se llena con handleEnqueueFlight, se vacía en handleProcessQueue
flightInEdit	Object | null	No null mientras usuario edita, null cuando cancela/guarda
processingQueue	boolean	true mientras se visualizan steps, false al terminar
stopProcess	boolean	true cuando usuario cliquea Stop, cancela visualización

7. CONVENCIONES Y PATRONES

Patrón	Ejemplo	Propósito
setLoading	setLoading(btn, true, "Cargando…")	Prevenir doubleclicking durante fetch
showToast	showToast(msg, "success")	Notificación nobloqueante
requestAnimationFrame	await new Promise(r => requestAnimationFrame(r))	Esperar layout browser antes de resize
confirm()	if (!confirm("¿Continuar?")) return;	Confirmación segura antes de operaciones destructivas
try/catch/finally	Todas las funciones async setLoading(false) en finally por si hay error	Garantizar limpeza de estado

Manual Técnico: api-client.js

Archivo: frontend/js/api-client.js

Cliente HTTP para comunicación entre frontend JavaScript y backend Flask - Wrapper de fetch() con manejo centralizado de errores.

1. PROPÓSITO GENERAL

api-client.js es el **módulo de comunicación HTTP** entre el frontend JavaScript y el backend Flask. Funciona como:

Capa de abstracción para todas las peticiones HTTP REST
Wrapper centralizado de fetch() con manejo de errores consistente

Punto único de configuración para la URL base de la API
Validación básica de respuestas y errores HTTP

2. CONFIGURACIÓN BASE

```
const API_BASE_URL = "http://localhost:5000/api/tree";
```

Descripción: URL base para todas las peticiones

Para cambiar servidor: Modificar este valor según la URL del backend

3. FUNCIÓN BASE: apiFetch()

Firma

```
async function apiFetch(endpoint, method = "GET", data = null)
```

Parámetros

Parámetro	Tipo	Obligatorio	Descripción
endpoint	string	-- Sí	Ruta relativa a API_BASE_URL (ej: "/insert")
method	string	-- No	Método HTTP (default: GET)
data	object	-- No	Body a enviar en JSON

Flujo de ejecución

1. Construye objeto options con método y headers
2. Si hay data, lo convierte a JSON y lo agrega al body
3. Realiza fetch() a URL base + endpoint
4. Parsea respuesta como JSON
5. Si response.ok === false: Lanza Error
6. Si response.ok === true: Retorna objeto JSON

4. FUNCIONES PRINCIPALES AGRUPADAS

Operaciones del Árbol AVL

Función	Método	Endpoint	Descripción
getTree()	GET	/	Obtener árbol actual del servidor
createTree(data)	POST	/create	Crear árbol desde archivo JSON
insertFlight(flight)	POST	/insert	Insertar nuevo vuelo
updateFlight(code, data)	PUT	/update/{code}	Actualizar vuelo existente
searchFlight(code)	GET	/search/{code}	Buscar vuelo por código
deleteFlight(code)	DELETE	/delete/{code}	Eliminar solo el nodo
cancelSubtree(code)	DELETE	/cancel/{code}	Eliminar nodo + subrama
resetTree()	DELETE	/reset	Limpiar árbol completo

Historial y Deshacer

Función	Método	Endpoint
undoAction()	POST	/undo

Exportación y Versioning

Función	Método	Endpoint
exportTree()	GET	/export
saveVersion(name)	POST	/version/save
loadVersion(name)	POST	/version/load/{name}
getVersions()	GET	/version

Cola de Inserción (Requisito 3)

Función	Parámetro	Endpoint
enqueueFlight(flight)	flight object	POST /queue/enqueue
processQueue()	—	POST /queue/process

Métricas y Auditoría

Función	Endpoint	Descripción
getMetrics()	GET /metrics	Estadísticas del árbol (altura, nodos, rotaciones)
auditAVL()	GET /auditory	Verificar propiedades AVL

Modo Estrés

Función	Endpoint
activateStress()	POST /stress/activate
deactivateStress()	POST /stress/desactivate
rebalanceStress()	POST /stress/rebalance

Rentabilidad y Profundidad

Función	Endpoint	Descripción
deleteLowestProfit()	POST /profit/deleteLowest	Eliminar vuelo menos rentable
setDepthLimit(limit)	POST /penalty/setDepthLimit	Establecer límite de profundidad

Ciudades

Función	Endpoint
getCities()	GET /cities

5. EJEMPLO DE USO

```
// Patrón básico try-catch
try {
    const response = await insertFlight(flightData);
    console.log("Vuelo insertado:", response.data);
    currentTree = response.data.tree;
    updateUI();
} catch (error) {
    console.error("Error:", error.message);
    showToast(error.message, "error");
}
```

6. CÓDIGOS HTTP MANEJADOS

```
2xx (200-299): Success - retorna JSON

4xx (400-499): Error del cliente - lanza Error

5xx (500-599): Error del servidor - lanza Error
```

7. PATRÓN DE RESPUESTA

```
Todas las respuestas exitosas siguen este formato:

{
    "data": { /* datos del servidor */ },
    "message": "Mensaje descriptivo"
}
```

8. ERRORES COMUNES Y SOLUCIONES

CORS Error
Causa: Backend no permite las peticiones desde este origen
Solución: Verificar configuración CORS en Flask backend
Network Error
Causa: Servidor no está corriendo o URL es incorrecta
Solución: Verificar API_BASE_URL y estado del backend
JSON Parse Error
Causa: Respuesta no es JSON válido
Solución: Revisar logs del servidor backend

Manual Técnico: tree-visualizer.js

Archivo: frontend/js/tree-visualizer.js

Componente de renderizado gráfico de árboles AVL en CanvasHTML5 con soporte para zoom, pan e interactividad.

1. PROPÓSITO GENERAL

TreeVisualizer es una **clase de renderizado gráfico** que transforma estructuras de datos de árbol (JSON) en visualizaciones interactivas en HTML5 Canvas. Proporciona:

Renderizado visual de nodos y aristas del árbol AVL/BST
Zoom dinámico (20% de incremento/decremento)
Pan interactivo (arrastrar el árbol con mouse)
Codificación de colores para diferentes estados de nodos
Estadísticas (altura, hojas, raíz)
Compatibilidad con CommonJS/Node.js además de browser

2. INICIALIZACIÓN

Constructor

```
const visualizer = new TreeVisualizer(canvasId, options = {})
```

Parámetros

Parámetro	Tipo	Default	Descripción
canvasId	string	(requerido)	ID del elemento <canvas>
options.treeType	string	"avl"	"avl" | "bst" - Mostrar balance factors
options.nodeRadius	number	28	Radio del círculo del nodo (px)
options.xSpacing	number	65	Espaciado horizontal entre nodos (px)
options.ySpacing	number	85	Espaciado vertical entre niveles (px)

Ejemplo de uso

```
// Inicialización básica
const visualizer = new TreeVisualizer('tree-canvas');

// Con opciones personalizadas
const visualizer = new TreeVisualizer('tree-canvas', {
    treeType: 'avl',
    nodeRadius: 35,
    xSpacing: 80,
    ySpacing: 100
});
```

3. FUNCIONES PRINCIPALES

draw(tree)

Propósito: Dibujar árbol completo en canvas

Parámetro: tree (object) - Árbol en formato JSON del backend

Retorna: void

Flujo:

1. Guarda referencia this.currentTree = tree
2. Calcula layout (posiciones X/Y de cada nodo)
3. Limpia canvas
4. Dibuja aristas (edges) primero
5. Dibuja nodos (circles) encima

Funciones de Navegación

zoomIn()

Propósito: Ampliar vista 20% (multiplicar zoom por 1.2)

Límite máximo: 500% (5.0x)

zoomOut()

Propósito: Reducir vista 20% (dividir zoom entre 1.2)

Límite mínimo: 20% (0.2x)

`resetView()`

Propósito: Restaurar zoom (1.0x) y pan (0, 20)

markConflicts(tree, conflictNodes)

Propósito: Colorear nodos en rojo si están desbalanceados

Parámetros:

- tree (object) - Árbol actual
- conflictNodes (Array<string>) - Códigos de nodos conflictivos

getStats(node)

Propósito: Obtener estadísticas generales del árbol

Retorna:

```
{
    root: "AA001",      // Código de la raíz
    depth: 5,           // Altura del árbol
    leaves: 8           // Cantidad de hojas
}
```

4. PALETA DE COLORES

Estado	Color	Hex	Significado
Normal	Turquesa	#4ECDC4	Vuelo estable
Promoción	Verde	#51CF66	Oferta/descuento activo
Alerta	Rojo	#FF6B6B	Problema táctico
Crítico	Amarillo	#FFD93D	Profundidad excesiva o conflictivo
Arista	Gris	#94a3b8	Línea de conexión
Borde	Oscuro	#1e293b	Outline del nodo

5. EVENTOS DE MOUSE Y ZOOM

Movimiento de mouse (Pan)

- **mousedown:** Iniciar arrastre (cursor → "grabbing")
- **mousemove:** Actualizar pan mientras se arrastra
- **mouseup/mouseleave:** Terminar arrastre (cursor → "grab")

Rueda del mouse (Scroll Zoom)

- ↑ Scroll arriba (deltaY negativo) → Zoom in 12%
- ↓ Scroll abajo (deltaY positivo) → Zoom out 10%
- Limitado a: [0.2x, 5x]

Cursor dinámico

- "grab" cuando está sobre canvas (listo para arrastrar)
- "grabbing" mientras se arrastra

6. FORMATO DE ÁRBOL ESPERADO (JSON)

```
{
    codigo: "AA001",
    origen: "Bogotá",
    destino: "Medellín",
    horaSalida: "10:30",
    precioBase: 250000,
    pasajeros: 180,
    promocion: true,
    alerta: false,
    prioridad: 1,
    esCritico: false,
    factorEquilibrio: 0,

    // Estructura binaria
    izquierdo: { ... },
    derecho: { ... }
}
```

7. EJEMPLO DE USO

```
// 1. Crear instancia
const visualizer = new TreeVisualizer('tree-canvas', {
    treeType: 'avl',
    nodeRadius: 30,
    xSpacing: 75,
    ySpacing: 90
});
// 2. Dibujar árbol
const response = await getTree();
visualizer.draw(response.data);
```

```
// 3. Interacciones de usuario
document.getElementById('btn-zoom-in').addEventListener('click', () => {
    visualizer.zoomIn();
});

// 4. Visualizar pasos de inserción
const steps = response.steps;
for (const step of steps) {
    visualizer.markConflicts(step.tree, step.imbalancedNodes);
    await sleep(1500);
}

// 5. Obtener estadísticas
const stats = visualizer.getStats();
console.log(`Altura: ${stats.depth}, Hojas: ${stats.leaves}`);
```